

Multilayer Perceptrons

from the XOR Problem to Backpropagation

why one line is not enough, network notation, counting parameters, and gradients flowing backward

Name

Branch / Year

Date

BY THE END OF THIS SHEET YOU CAN

- ☐ Show why a single perceptron **cannot** solve the XOR problem.
- ☐ Explain what a **Multilayer Perceptron (MLP)** is and how a layer computes.
- ☐ Read and write the **weight and bias notation** for any layer.
- ☐ **Count the total parameters** of a network from its shape.
- ☐ Build XOR from an **OR** neuron and an **AND** neuron inside one MLP.
- ☐ Run **backpropagation** by hand on a small network with squared-error loss.

PART 1 THE XOR PROBLEM

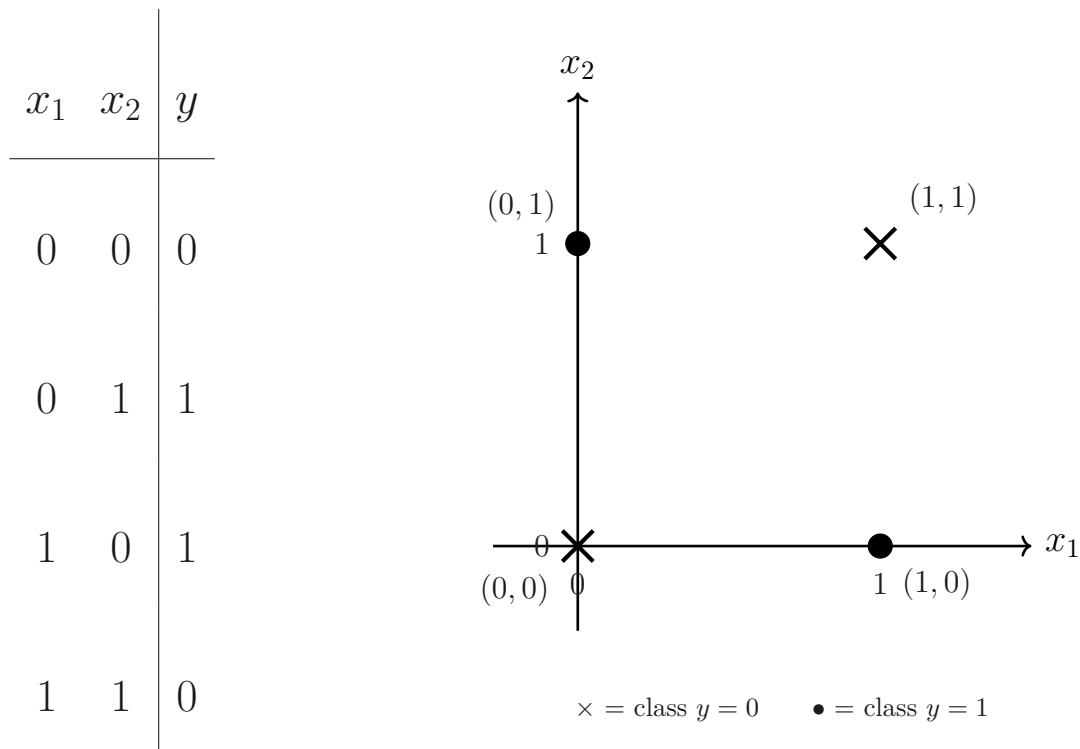
1 Can we solve the XOR problem using a perceptron?

THE OPENING QUESTION

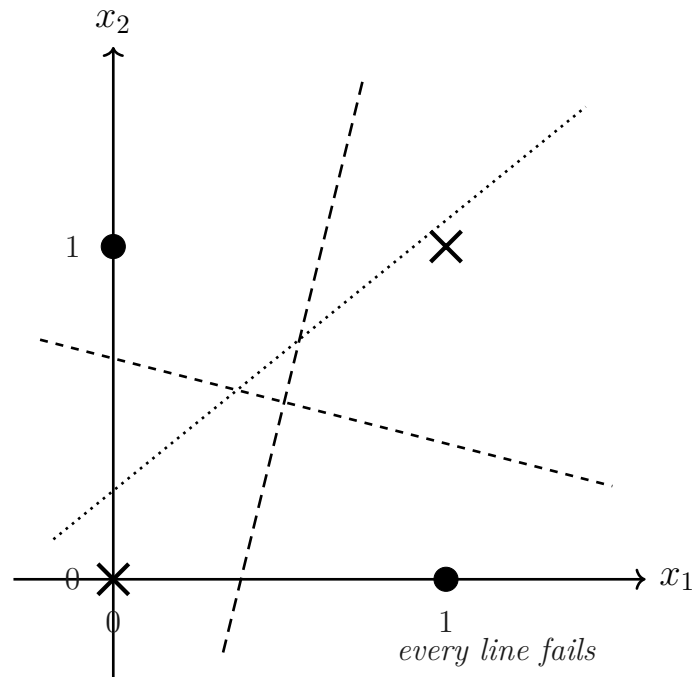
Can a single perceptron solve the XOR problem?

Recall XOR: the output is 1 only when the two inputs are *different*. Before reading on, guess: ☐ Yes ☐ No

Here is the XOR truth table and its picture. Each input pair (x_1, x_2) is a point on the plane; its symbol is the label.



A perceptron can only plot a straight line. Try fitting a straight line in this and separate the data points.



$\times$ = class $y = 0$ $\bullet$ = class $y = 1$ (dashed lines: candidate separators)

Read the picture

No matter how you rotate or shift the line, one $\times$ always ends up on the same side as the $\bullet$ points, or one $\bullet$ lands among the $\times$ s. The two classes *cannot* be separated by a single straight line.

The conclusion

Because XOR **cannot be separated by a straight line**, we call it **non-linearly separable** (“non-linear”) data. A single perceptron is a linear classifier, so it **cannot** solve XOR.

Quick check

In one sentence, why does the perceptron fail on XOR?

PART 2 THE MULTILAYER PERCEPTRON

2 How do we classify non-linear data?

THE NEXT QUESTION

If one line cannot separate XOR, how do we classify non-linear data?

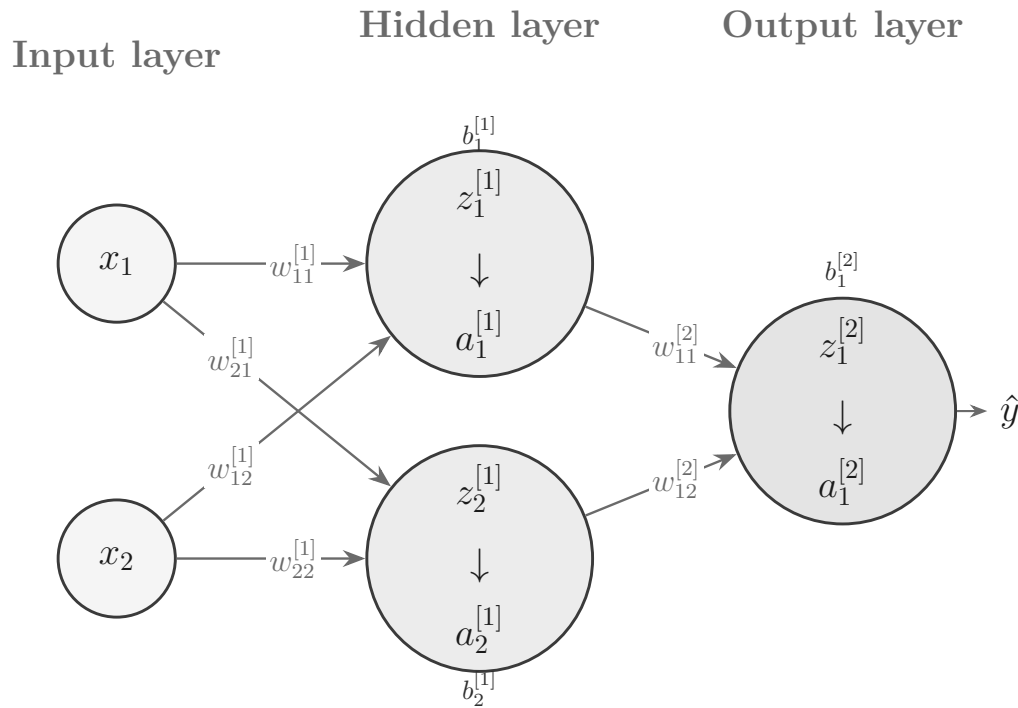
Answer: stack neurons into layers so the network can bend many lines together. This is the **Multilayer Perceptron (MLP)**.

What is an MLP?

An MLP is a neural-network architecture made of **many neurons arranged in layers**: an **input layer** (just holds the inputs), one or more **hidden layers**, and an **output layer**. Every hidden neuron and every output neuron does the **same two-step computation** — first a **linear combination**, then an **activation function**:

$$z = \underbrace{w_1x_1 + w_2x_2 + \cdots + b}_{\text{linear combination}}, \quad a = \underbrace{\phi(z)}_{\text{activation}} .$$

Below is a **2–2–1** MLP: two inputs, one hidden layer of two neurons, one output neuron. Each neuron shows its two steps — first z (the linear combination), then $a = \phi(z)$.



What each hidden and output neuron computes

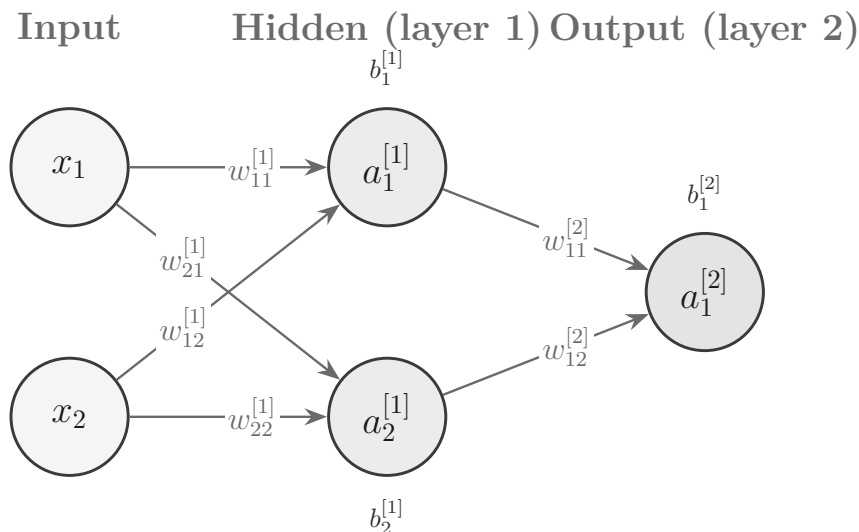
$$\begin{aligned}
 z_1^{[1]} &= w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + b_1^{[1]}, & a_1^{[1]} &= \phi\left(z_1^{[1]}\right), \\
 z_2^{[1]} &= w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + b_2^{[1]}, & a_2^{[1]} &= \phi\left(z_2^{[1]}\right), \\
 z_1^{[2]} &= w_{11}^{[2]}a_1^{[1]} + w_{12}^{[2]}a_2^{[1]} + b_1^{[2]}, & \hat{y} &= a_1^{[2]} = \phi\left(z_1^{[2]}\right).
 \end{aligned}$$

The input layer only *holds* x_1, x_2 — it does no computation.

PART 3 NOTATION OF WEIGHTS AND BIASES

3 Reading the labels: weight and bias notation

To talk about any network cleanly, we need a fixed way to name every weight and bias. Here is the same **2–2–1** MLP again, and next to it, exactly what each symbol means.



Decode every symbol

Weight $w_{jk}^{[\ell]}$: the weight in **layer** ℓ that goes *into* neuron j of that layer, *from* neuron k of the previous layer.

$w_{jk}^{[\ell]} \implies$

- ℓ = which layer,
- j = destination neuron (this layer),
- k = source neuron (previous layer).

Bias $b_j^{[\ell]}$: the bias added inside neuron j of layer ℓ . Every neuron has *one* bias.

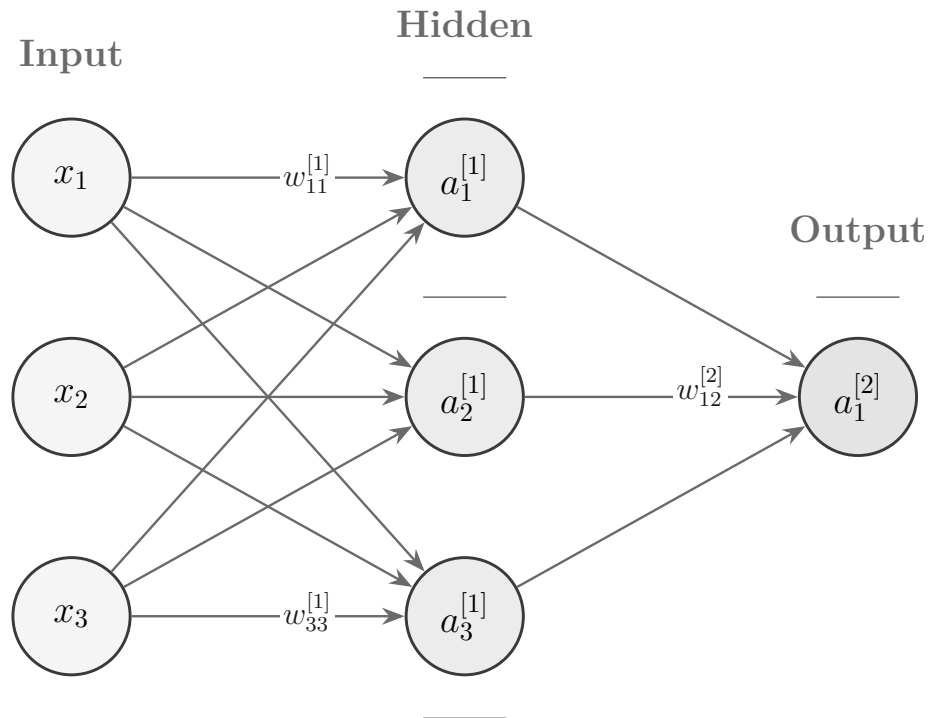
Activation $a_j^{[\ell]}$: the output of neuron j in layer ℓ after the activation function.

Worked reading

$w_{21}^{[1]}$ is read as: layer 1, into hidden neuron 2, from input 1. $w_{12}^{[2]}$ is: layer 2, into output neuron 1, from hidden neuron 2. $b_2^{[1]}$ is: the bias of hidden neuron 2.

Activity 3 — label the 3–3–1 network

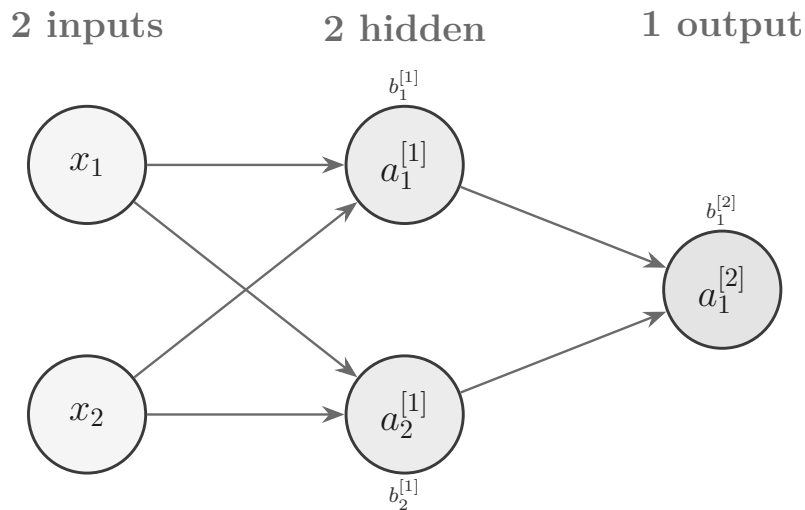
Below is a **3–3–1** MLP. A few weights are filled in for you. **Write the correct notation** ($w_{jk}^{[\ell]}$ or $b_j^{[\ell]}$) directly on each blank edge and bias slot in the diagram.



PART 4 COUNTING THE PARAMETERS

4 How many parameters does a network have?

The parameters a network *learns* are exactly its **weights** and **biases**. Let us count them for the **2–2–1** network from before.



Count layer by layer

Layer	Weights	Biases	Subtotal
Hidden (2 inputs $\rightarrow$ 2 neurons)	$2 \times 2 = 4$	2	6
Output (2 hidden $\rightarrow$ 1 neuron)	$2 \times 1 = 2$	1	3
Total	6	3	9

General formula

For a layer with n_{in} incoming units and n_{out} neurons:

$$\text{parameters in a layer} = \underbrace{n_{\text{in}} \times n_{\text{out}}}_{\text{weights}} + \underbrace{n_{\text{out}}}_{\text{biases}}.$$

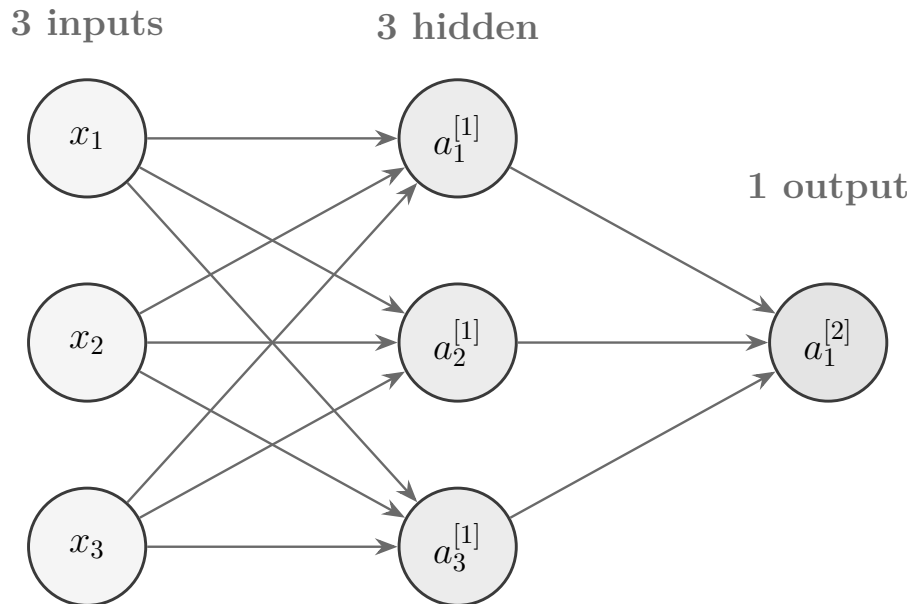
Add this up over every layer to get the network total.

Remember this

The input layer has NO bias. Inputs are just given values — they are not neurons, so they carry no weight and no bias. Biases begin at the first hidden layer.

Activity 4 — count the 3–3–1 network

Here is a **3–3–1** MLP (3 inputs, 3 hidden neurons, 1 output). Count its parameters step by step.



Step 1 — Total weights

Hidden layer weights = _____ Output layer weights = _____

Total weights = _____

Step 2 — Total biases

Hidden layer biases = _____ Output layer biases = _____

Total biases = _____

Step 3 — Total parameters

Total parameters = total weights + total biases = _____

PART 5 MLP SOLVES XOR

5 Enough MLP theory — back to the big question! 😊

REMEMBER WHY WE STARTED

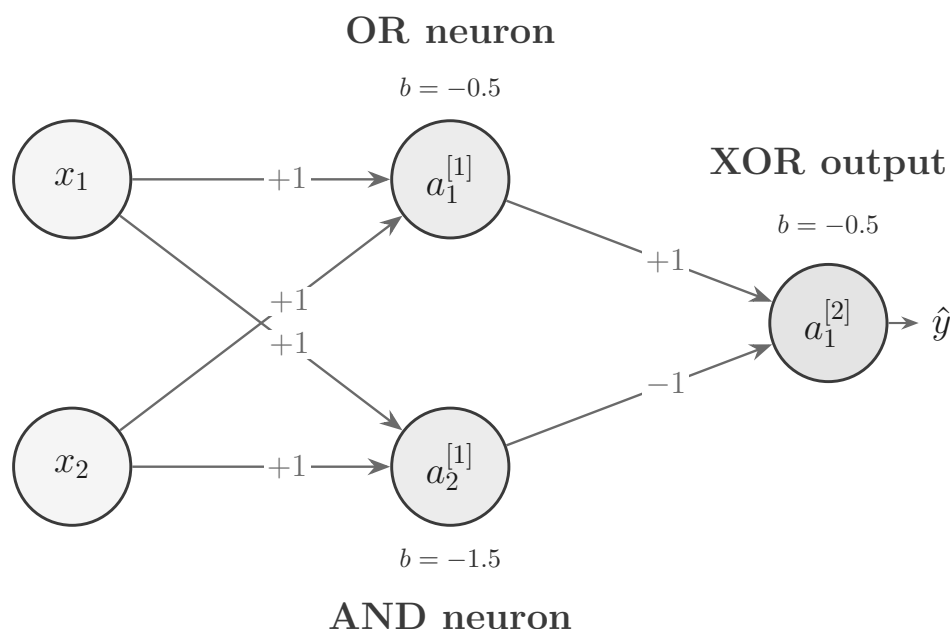
Enough learning MLP for a moment. 😊 The whole reason we built it was one stubborn question:

Can non-linear data — the XOR problem — be solved by an MLP?

Let us actually build it and find out.

5.1 The trick: one neuron does OR, the other does AND

XOR is true when the inputs *differ*. Notice: that is exactly “ x_1 OR x_2 is true, **but** x_1 AND x_2 is *not*.” So we let the hidden layer compute both gates, then the output combines them. We use a **step** activation: output 1 if $z \geq 0$, else 0.



What each neuron fires on

OR: $a_1^{[1]} = \text{step}(x_1 + x_2 - 0.5)$ (fires when at least one input is 1)

AND: $a_2^{[1]} = \text{step}(x_1 + x_2 - 1.5)$ (fires only when both inputs are 1)

XOR: $\hat{y} = \text{step}(a_1^{[1]} - a_2^{[1]} - 0.5)$ (fires when OR is on but AND is off)

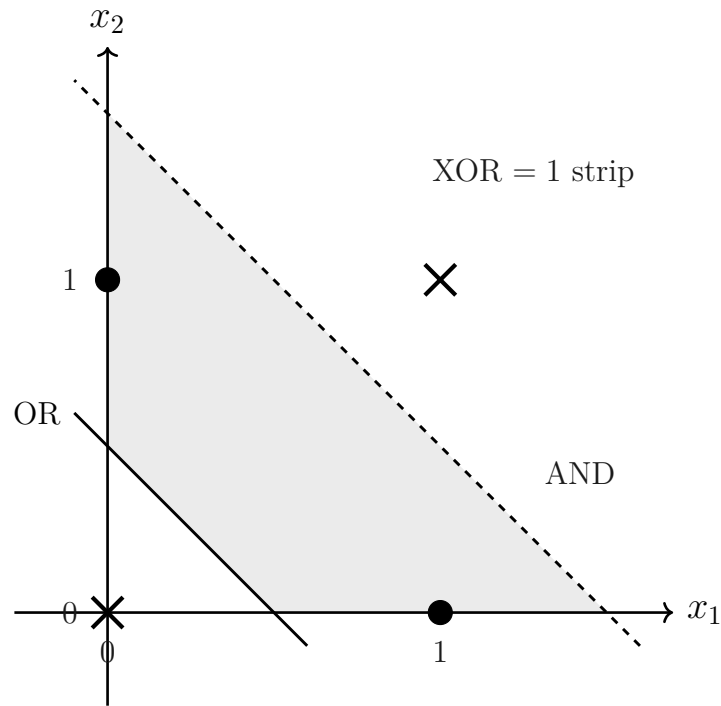
5.2 Check it with the truth table

x_1	x_2	$a_1^{[1]}$ (OR)	$a_2^{[1]}$ (AND)	$\hat{y} = \text{OR} - \text{AND}$ (XOR)
0	0	0	0	$\text{step}(0 - 0 - 0.5) = 0$
0	1	1	0	$\text{step}(1 - 0 - 0.5) = 1$
1	0	1	0	$\text{step}(1 - 0 - 0.5) = 1$
1	1	1	1	$\text{step}(1 - 1 - 0.5) = 0$

The last column is exactly XOR. **The MLP solved it.**

5.3 Why it works — see it on the graph

Each hidden neuron draws *one* line. The **OR** line and the **AND** line are parallel; XOR is the **strip between them**. One line was never enough — two lines are.



$\times$ = class $y = 0$ $\bullet$ = class $y = 1$ (solid: OR line dashed: AND line)

The answer

Yes — an MLP can solve XOR. By combining an OR neuron and an AND neuron, the hidden layer turns a non-linearly-separable problem into a linearly-separable one for the output neuron.

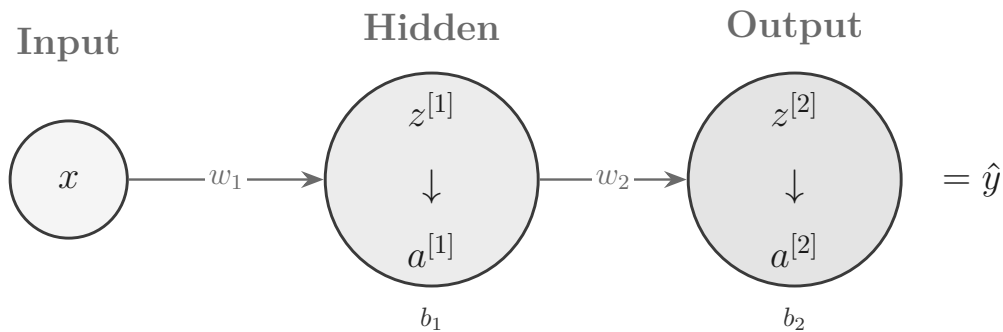
PART 6 BACKPROPAGATION IN AN MLP

6 How does the network learn? Backpropagation

We use a **1–1–1** network — one input, one hidden neuron, one output — with the **sigmoid** activation and the squared-error loss

$$\mathcal{L} = \frac{1}{2} (\hat{y} - y)^2$$

where $\hat{y}$ is the network's prediction and y is the true label.

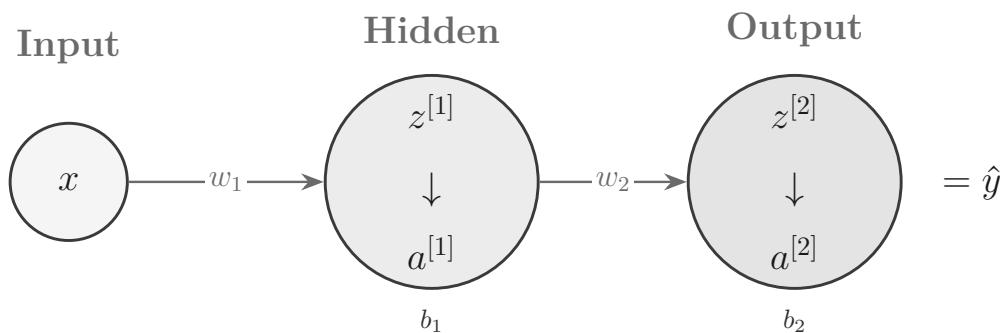


6.1 Forward pass (compute the prediction)

$$z^{[1]} = w_1 x + b_1, \quad a^{[1]} = \text{sigmoid}(z^{[1]}), \quad z^{[2]} = w_2 a^{[1]} + b_2, \quad \hat{y} = a^{[2]} = \text{sigmoid}(z^{[2]}).$$

6.2 Backward pass — the idea

The forward pass gave us a prediction $\hat{y}$ and a loss. **Backpropagation** finds, for each weight, how to change it so the loss gets smaller. We work from the output back to the input. Use the network below as your reference, then work out the full backward pass yourself on the next page.



Backward Propagation — Do It Yourself
